Inside a Coding-Agent Harness

A repository repair, the execution loop, and the evidence for a change

CMSC 848N Fall 2026
W7-L1 October 15 Furong Huang, Seungjae Lee

The coding-agent harness

The harness builds model inputs, executes actions, returns observations, and decides when execution stops.

We will follow one recorded repair and distinguish the model's decisions from the tools' behavior.

We will inspect the implementation and test a choice: what should happen when an edit fails lint?

Next lectures: training trajectories, then automatic harness optimization.

The task: read floating-point pixel data

pydicom issue #1457 reports a failure when reading FloatPixelData.

```
ds.BitsAllocated = 32
ds.SamplesPerPixel = 1
ds.Rows = 5
ds.Columns = 5
ds.PhotometricInterpretation = 'MONOCHROME2'

pixel_array = np.zeros((5, 5), dtype=np.float32)
ds.FloatPixelData = pixel_array.flatten().tobytes()

result = np.array_equal(ds.pixel_array, pixel_array)
```

Expected here: reading the bytes recovers the same 5 × 5 array.

pydicom #1457; SWE-agent trace, step 2. Excerpt omits imports and transfer-syntax setup.

The reproduction identifies a required attribute

Recorded action 3

```
$ python reproduce_bug.py

.../pixel_data_handlers/numpy_handler.py, line 293
    raise AttributeError(...)

AttributeError: Unable to convert the pixel data
as the following required elements are missing
from the dataset: PixelRepresentation
```

The traceback names the handler and the missing attribute.

SWE-agent trace, step 3. Traceback excerpt with line wrapping.

The traceback identifies which file to open

Recorded action 4: find_file "numpy_handler.py"

pydicom/overlays/numpy_handler.py
pydicom/pixel_data_handlers/numpy_handler.py
pydicom/waveforms/numpy_handler.py

open pydicom/pixel_data_handlers/numpy_handler.py 293

The viewer supplies line numbers and a bounded window around the failure.

SWE-agent trace, steps 4–5. Repository prefix omitted from search results.

The handler requires integer-pixel metadata

PixelRepresentation marks signed or unsigned integer pixels. Floating-point pixels should not require it.

```
287  required_elements = [
288      'BitsAllocated', 'Rows', 'Columns', 'PixelRepresentation',
289      'SamplesPerPixel', 'PhotometricInterpretation'
290  ]
291  missing = [elem for elem in required_elements if elem not in ds]
292  if missing:
293      raise AttributeError(
294          'Unable to convert the pixel data ...'
295          + ', '.join(missing)
296      )
```

Attempt 2 replaces lines 287–295 inclusive and supplies its own closing parenthesis. Which original line remains?

Why the second edit fails

Recorded second edit attempt: edit 287:295

Proposed replacement ends with

```
raise AttributeError(
    "Unable to convert …"
    + ", ".join(missing)
)
```

E999 SyntaxError: unmatched ')'

Recorded candidate file ends with

```
294  raise AttributeError(
295      "Unable to convert …"
296      + ", ".join(missing)
297  )
298  )
```

Original line 296 survives as candidate line 298.

SWE-agent trace, action 7. Exception text shortened. Original line numbers retained in labels.

Three rejected edits, then an accepted patch

The tool writes the proposed edit, runs its checks, then restores the backup on failure.

Attempt	Replacement range	Tool feedback
1	287:295	unmatched ']'
2	287:295	unmatched ')'
3	287:295	unmatched ')' (same edit repeated)
4	287:296	File updated

After each rejection, the next model turn sees the restored file.

SWE-agent trace, steps 6–9; recorded edit-tool implementation.

The accepted patch changes one requirement

Semantic core of the submitted patch, reformatted for readability

```
required_elements = [
    'BitsAllocated', 'Rows', 'Columns',
    'SamplesPerPixel', 'PhotometricInterpretation'
]
if 'PixelData' in ds:
    required_elements.append('PixelRepresentation')
```

FloatPixelData bypasses this added condition. Integer PixelData still triggers it.

The agent reruns its reproduction and submits

Recorded actions 10–12

```
$ python reproduce_bug.py
Script completed successfully, no errors. Result: True

$ rm reproduce_bug.py

$ submit
```

Recorded status: submitted. The trace contains no test-suite run.

What has the agent actually verified?

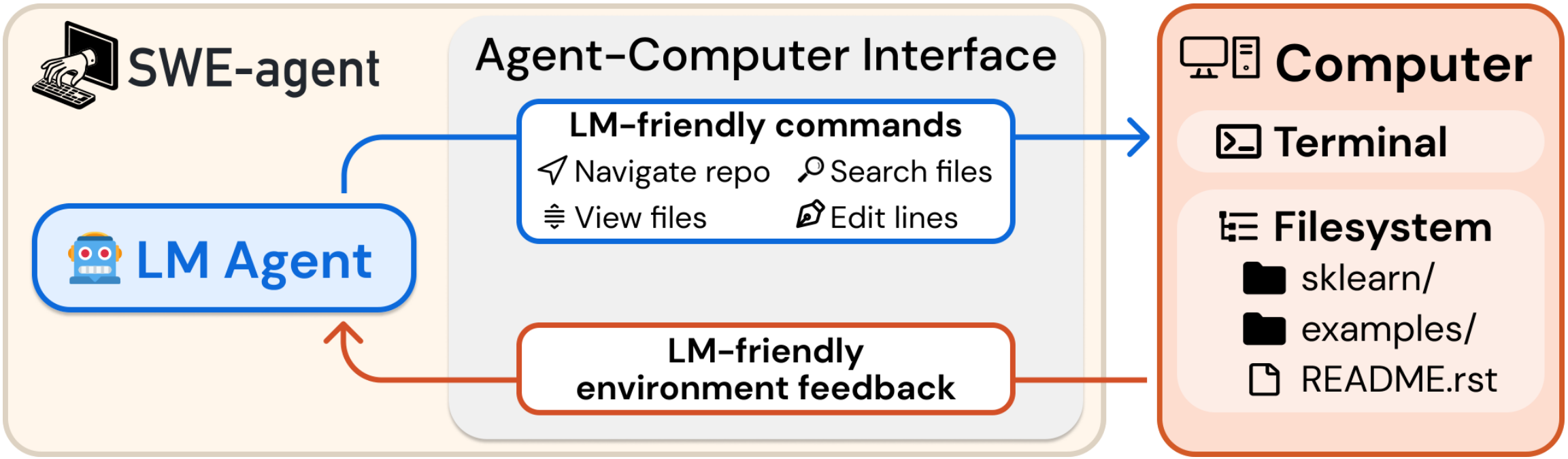
Observation	What it supports	What remains open
Edit accepted	Selected syntax checks passed	Behavior of the edited code
Reproduction prints True	Values and shape agree for this zero array	Output dtype, nonzero inputs, other attributes
Agent submits	The controller ended the attempt	Held-out test success

The human PR also checks BitsStored and PlanarConfiguration. This reproduction does not cover those cases.

SWE-agent trace; pydicom PR #1458; SWE-bench task fixture.

The interface determines what reaches the model

SWE-agent exposes search, file viewing, editing, and execution through an agent-computer interface.



Our trace shows both sides: useful feedback and a replacement-range error.

Yang et al., SWE-agent, Figure 1 (2024). CC BY 4.0. Original figure.

A model call inside an execution loop

$$c_t = C_h(q, \tau_t, z_t), \qquad u_t \sim \pi_\theta(\cdot \mid c_t)$$

$$a_t = D_h(u_t, z_t), \qquad (s_{t+1}, o_{t+1}) \sim E_h(\cdot \mid s_t, a_t)$$

q: task; τ_t: interaction history; z_t: harness state; s_t: environment state.

C_h constructs context. D_h parses tool requests. E_h executes them, including any checks and rollback.

The harness records feedback and updates its state before the next call.

The second edit through the execution loop

Part of the loop	This recorded action
Context C_h	Repair task, earlier feedback, numbered code, current file view
Parser D_h	Parse response u_t into edit request a_t: edit 287:295.
Execution E_h	Write the candidate → check → restore the backup on failure.
Observation o_{t+1}	Return the syntax error and candidate/original excerpts.

The target file returns to its prior contents. History gains the failed action and its feedback.

Instructional mapping of SWE-agent trace, action 7. The parser accepts the request; execution triggers rollback.

The execution loop in mini-SWE-agent

Different implementation: general Bash tool calls, with no automatic edit rollback.

```
while True:
    try:
        check_limits()
        response = model.query(messages)
        messages.append(response)
        outputs = execute(response.tool_calls)
        messages.extend(format_observations(outputs))
    except StopOrRecover as event:
        messages.extend(event.messages)
    if has_exit_message(messages):
        break
```

One response may contain several actions. The pinned version uses tool calls.

mini-SWE-agent default.py, commit 04d809ce. MIT. Simplified pseudocode.

Error handling is part of the agent's behavior

Event	Controller behavior	Consequence
Malformed tool response	Corrective feedback, then a retry limit	Repeated format errors can end the run
Command timeout	Terminate it; keep partial output	Partial evidence may guide recovery
Cost limit reached	Check before the next model call	One call can overshoot the limit
Submission signal	End the attempt	Correctness still needs an evaluator

mini-SWE-agent, pinned agent and LocalEnvironment implementations. Commit 04d809ce.

File state and shell state persist differently

Action 1: start at /workspace, MODE unset

```
cd /workspace/project

export MODE=debug

echo checkpoint > progress.txt
```

Action 2: new shell with the same defaults

```
pwd

echo "$MODE"

cat /workspace/project/progress.txt
```

Action 2 prints /workspace, a blank line, then checkpoint. The file persists across shells.

Illustrative example based on mini-SWE-agent's fresh-subshell configuration. Not a recorded run.

A shell tool inherits the environment's authority

LocalEnvironment executes commands on the host. A classroom run should use an isolated workspace with explicit access limits.

Repository text and command output can contain instructions. They remain untrusted task data.

Network access, mounted files, credentials, and resource limits determine what a mistaken action can affect.

For our exercise: disposable repository, no host secrets, bounded runtime, controlled network access.

Implementation fact: mini-SWE-agent LocalEnvironment. Safeguards are course recommendations.

Output truncation changes the evidence

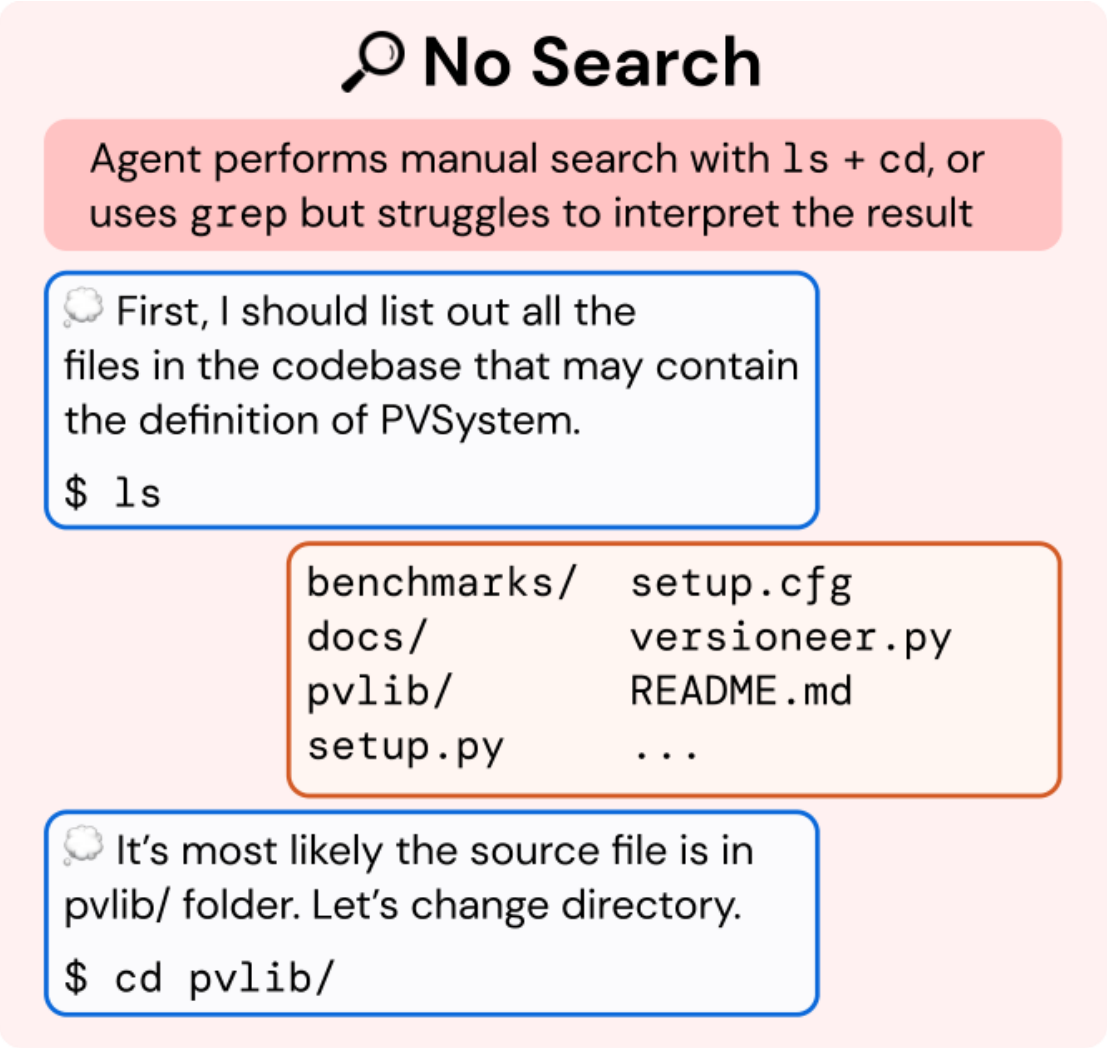
The pinned mini configuration retains the start and end of long output.

```
if len(output) >= 10_000:
    visible = (output[:5_000]
               + truncation_notice
               + output[-5_000:])
```

Units are characters. A failure in the middle may require a targeted command to recover.

mini.yaml, commit 04d809ce. Template behavior expressed as pseudocode.

Search with general shell commands



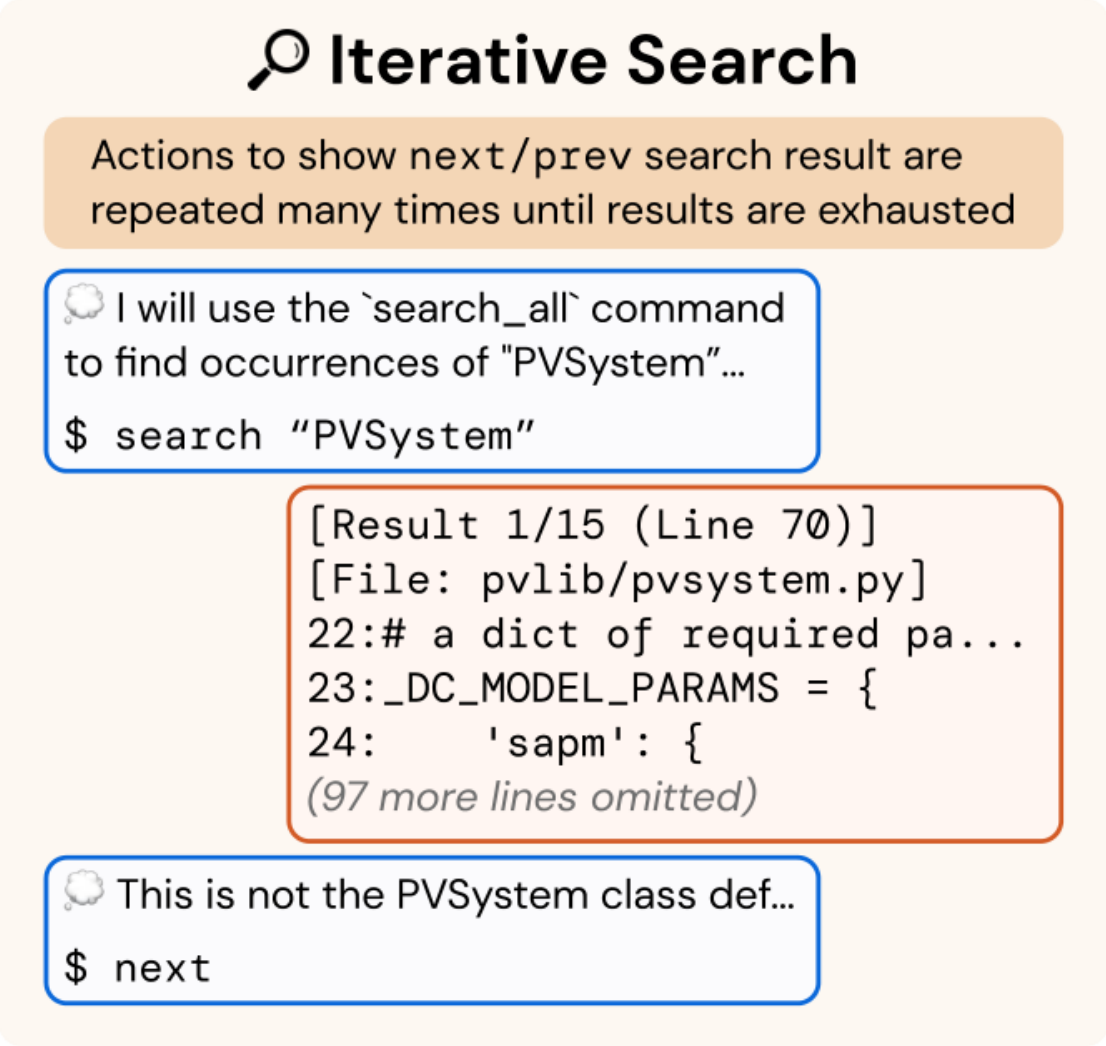
Search task: find the file defining class PVSystem.

The model chooses shell commands. Here, a directory listing leads to another navigation step.

Which observation would let the model locate PVSystem sooner?

Yang et al., SWE-agent, search-interface comparison (2024). CC BY 4.0. Enlarged source panel.

Iterative search reveals one match at a time



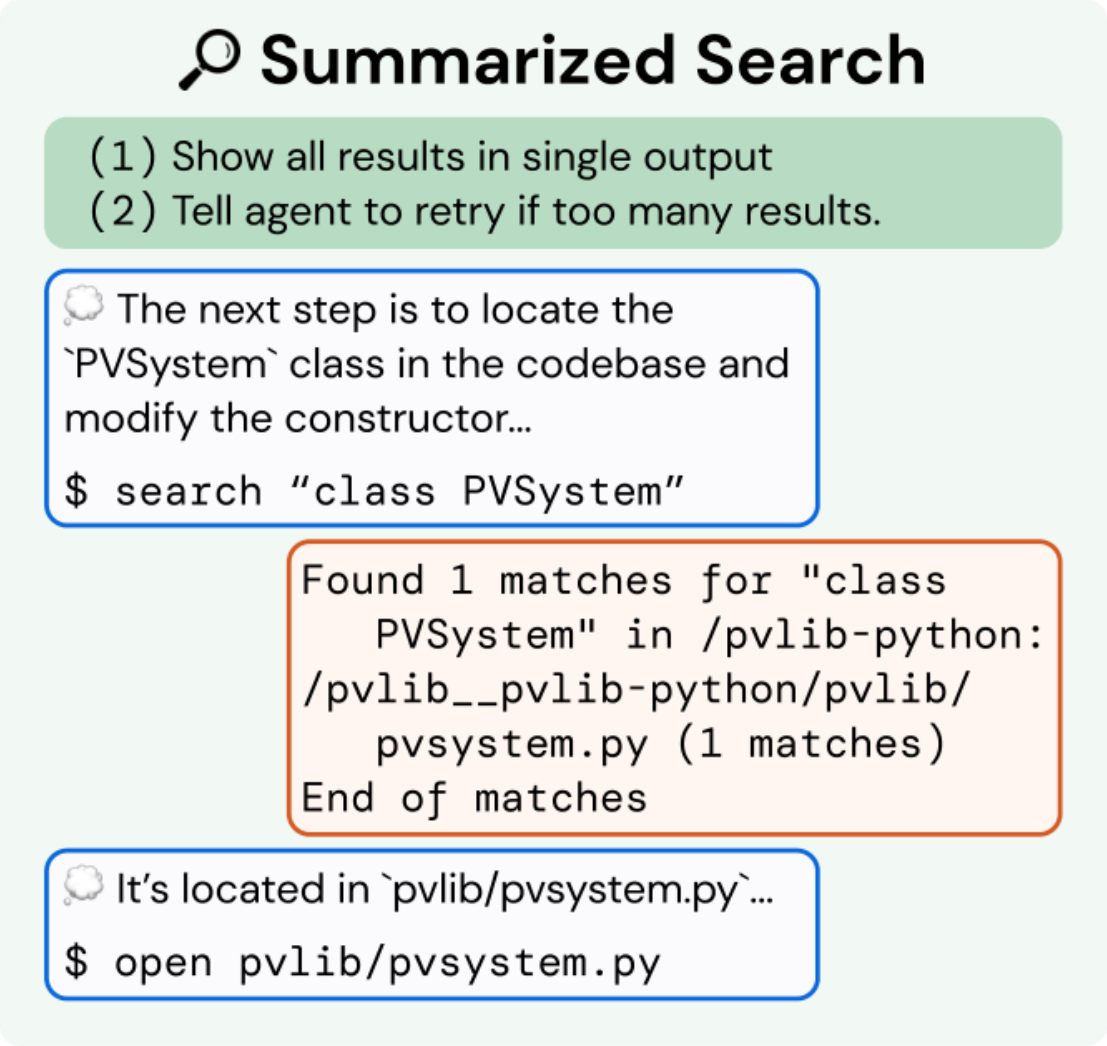
The interface returns a match and its local context.

The model may spend more actions asking for the next result.

How should the interface handle many plausible matches?

Yang et al., SWE-agent, search-interface comparison (2024). CC BY 4.0. Enlarged source panel.

Summarized search helps select a file



The interface reports a bounded overview before the file is opened.

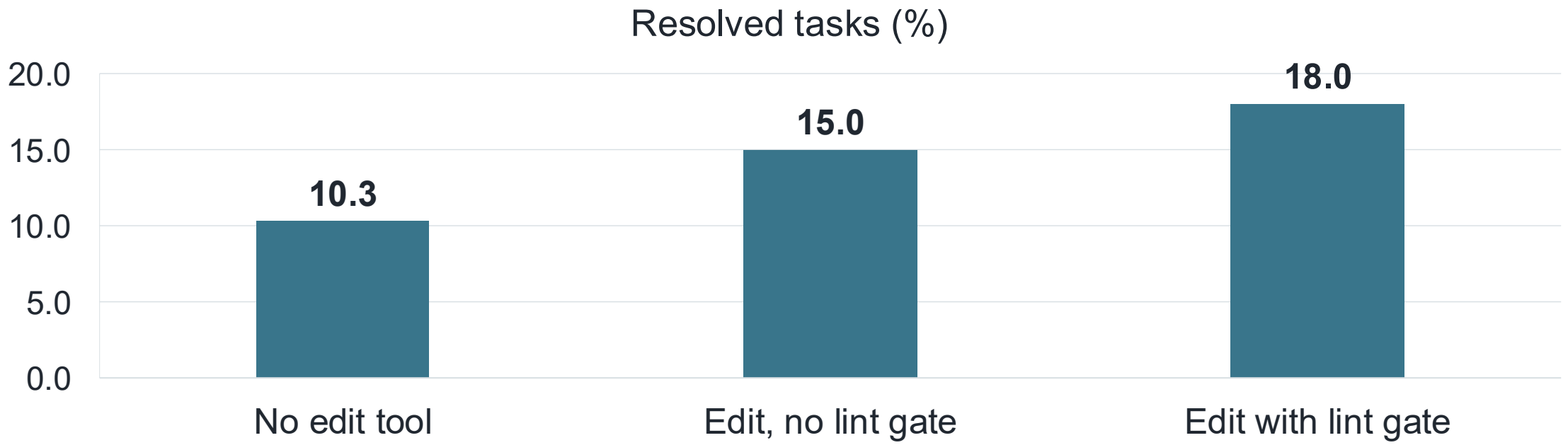
In the paper, more than 50 results triggers a request to narrow the query.

Would your preferred cutoff change with the model or token budget?

Yang et al., SWE-agent, search-interface comparison (2024). CC BY 4.0. Enlarged source panel.

SWE-agent's editing-tool ablation

SWE-bench Lite, 300 tasks, GPT-4 Turbo. Table 3 of SWE-agent.



18.0% versus 15.0% is a 3.0 percentage-point difference in this setting.

The comparison combines lint feedback and enforcement. It does not isolate rollback alone.

Yang et al., SWE-agent, Table 3 (2024), reported resolved rates. Historical experiment.

Editing and observation choices in the two harnesses

Component	Original SWE-agent	Pinned mini-SWE-agent
Actions	Specialized commands	General Bash tool calls
Editing	Line ranges and a lint gate	No equivalent automatic gate
Observations	Bounded search and file views	Configured output truncation
Model’s work	Specialized syntax and exact line ranges	Shell commands and explicit checks

Compare the work each interface asks the model to do, then measure the result.

SWE-agent paper and recorded tools; mini-SWE-agent commit 04d809ce.

A lint gate constrains edit order

Remove debug. Assume callers no longer pass it. This checker flags only undefined names.

```
# Initial code
def score(x, debug=False):
    if debug:
        print(x)
    return x

# First edit removes only the parameter
def score(x):
    if debug:    # undefined until the next edit
        print(x)
    return x
```

Remove the debug block first, or replace the complete function in one edit.

Constructed example inspired by the lint-gate limitation in SWE-agent, Figure 11.

What will lint enforcement change?

Both conditions use the same edit API. Source writes outside that API are blocked.

Condition	Lint feedback	Resulting file state
A: Keep	Same checker and format	Apply the edit and report that state
B: Restore	Same checker and format	Restore the prior file and report that state

Pairs, 4 minutes: predict success and cost for repairs and refactors. Give a concrete case and one essential control.

Course experiment motivated by SWE-agent's editor ablation and lint-gate limitation.

Matched tasks and measurable outcomes

Use the same model, task snapshots, visible tests, prompts, checker, and diagnostic format. Choose a primary resource budget.

Tune on separate tasks. Freeze both harnesses and define repair/refactor groups before evaluation.

Record final test success, actual cost, latency, and rejected edits. A rejection count alone cannot establish benefit.

Repeat every task in both conditions. Log edit-API bypass attempts and retain unsolved tasks.

Paired effects for repairs and refactors

$$\bar{Y}_i^{(h)} = \frac{1}{R}\sum_{r=1}^{R} V_i(s_{i,r,T}^{(h)})$$

$$\widehat{\Delta} = \frac{1}{N}\sum_{i=1}^{N}\left(\bar{Y}_i^{(B)} - \bar{Y}_i^{(A)}\right)$$

V_i is the held-out test criterion for task i. R runs per task, N tasks.

Compute Δ separately for repairs and refactors. Positive Δ favors restoring failed edits.

Report task-cluster uncertainty, actual cost, and latency for each group.

Instructional estimator for the proposed experiment; assumes a fixed evaluation protocol.

A matched probe after one failed edit

Fork the same task, history, files, harness state, and remaining budget before a preselected failing edit.

Apply it under A and B. Count turns to the next nonempty edit that passes lint.

Fix a cap K. Assign K to unfinished probes and report their fraction.

This endpoint can include reversions or whitespace changes. Report those cases and final test success separately.

Stopping status and verified task success

$$\sigma_h(\tau_t, z_t) \in \{\text{continue}, \text{submit}, \text{limit}, \text{error}\}$$

$$V(q, s_T) \in \{0, 1\}$$

σ_h determines whether execution continues. V scores the final state under a specified criterion.

Our pydicom trace reaches submit. Its reproduction passes. Broader test success remains unmeasured here.

What to inspect in a coding-agent harness

The action contract: what the model can request and how the tool changes the repository.

The observation contract: what feedback enters context and what the harness leaves out.

The stopping and evaluation rules: why execution ends and what evidence supports success.

A proposed harness change should name a failure it addresses and a comparison that could refute it.

Training data needs more than a submitted patch

Part of the record	What we can learn from it	Additional check
Failed edit and feedback	How the agent attempts recovery	Did the tool preserve the intended state?
Successful reproduction	A local behavioral check	Does the patch pass independent tests?
Final submitted patch	The proposed code change	Does it satisfy the task without regressions?

Next lecture: constructing executable tasks and deciding which trajectories provide useful supervision.

Interpretation of the recorded pydicom trace. Training choices are questions for the next lecture.

Readings and code to inspect

SWE-agent, Sections 3–5 and Table 3. Read the interface choices and their ablations.

mini-SWE-agent control-flow guide. Follow default.py into the execution environment and configuration.

The pydicom recorded trace. Compare its reproduction, submitted patch, and the human PR.

Exact source revisions, code excerpts, and reuse notices accompany these slides.

The same loop, now controlling a robot

	Software agent (pydicom trace)	Embodied coding agent
Action	Edit files, run shell commands	Write programs that call perception and control APIs
Observation	Traceback, test output, file view	Logs, camera images, depth, measured outcomes
Success check	Held-out tests	Task success in simulation or on the robot
Retry	Restore the backup, rerun the script	Reset the scene; physical attempts cannot be undone like an edit

Two papers, two answers to one question: when the robot fails, what evidence does the agent get? CaP-X, then Fly-by-Code.

Instructional comparison. Fu et al., CaP-X (ICML 2026); Lee et al., Fly-by-Code (2026 preprint).

CaP-X: a gym, a benchmark, an agent, and RL

CaP-Gym

Interactive environment: agents control robots by writing and running programs.

187 tasks: 7 Robosuite, 130 LIBERO-PRO, 50 BEHAVIOR

CaP-Bench

Studies three axes: abstraction level, single- vs multi-turn, and visual grounding.

12 models, 7 core tasks, 100 trials per tier

CaP-Agent0

Training-free harness: visual differencing, a skill library, parallel reasoning.

Built from the benchmark's failure modes

CaP-RL

RL with verifiable rewards on the coding agent itself (GRPO).

Qwen2.5-Coder-7B; code transfers to a real Franka arm

One turn: observe, write a Python program, execute it to completion, read the result. The agent calls perception and control primitives; it does not emit joint commands.

https://capgym.github.io/

Fu et al., CaP-X, ICML 2026 (arXiv 2603.22435). Counts as reported in Sections 1-5.

What the agent sees after a failure

Tier	Feedback the agent receives	Reported effect
M1	stdout and stderr from the sandbox	Improves most models; agents add prints to expose state
M2	Raw RGB image in the context	Lowers success compared with M1
M3	Text from a Visual Differencing Module: scene, changes, task done?	Improves success for closed and open models
M4	M3 feedback, low-level primitives with usage examples	Reaches parity with high-level multi-turn (M3)

Seeing more is not seeing usefully: the same image hurt as pixels and helped as text.

Fu et al., CaP-X, Section 3.2-3.3 (Takeaway 3), Figures 5-6. Qualitative summary of reported trends.

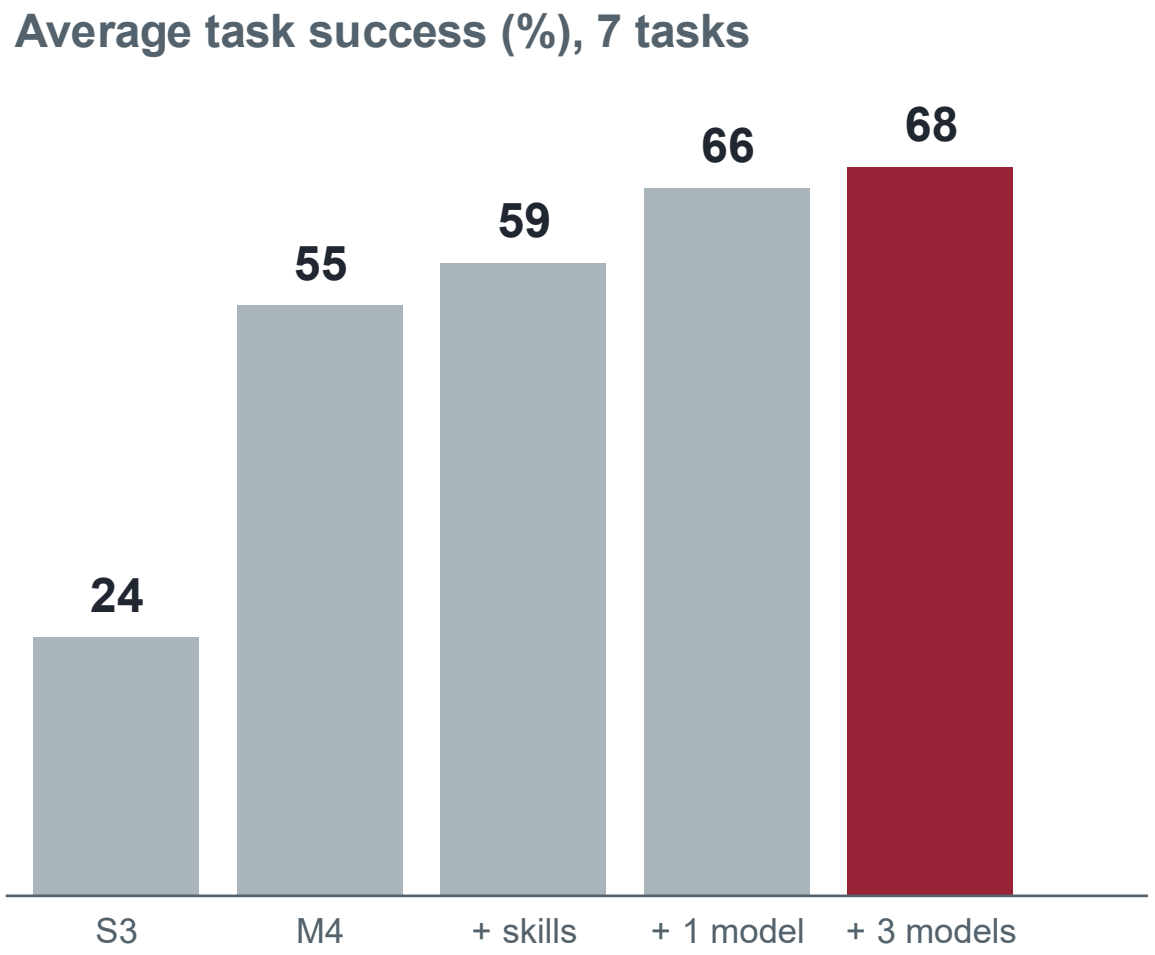
CaP-Agent0 adds three harness components

Visual differencing
A VLM turns each image change into text for the coding agent.

Skill library
9 verified, task-agnostic helpers extracted from successful low-level rollouts.

Parallel reasoning
9 queries to one model, or 3 each to 3 models; a central agent merges them.

CaP-RL: GRPO on Qwen2.5-Coder-7B; real Franka arm lifts the cube 84% and stacks 76%.



Fu et al., CaP-X, Section 4, Figure 8. Gemini-3-Pro base, 7 Robosuite tasks, 100 trials per task; CaP-RL from Section 5.

Where passive feedback runs out

Missing outcome observation

Partial observability

The egocentric camera is coupled to the flight path, so the end state can fall outside the view.

An empty gripper after release does not show that the object reached the target.

Unresolved failure cause

Limited interaction robustness

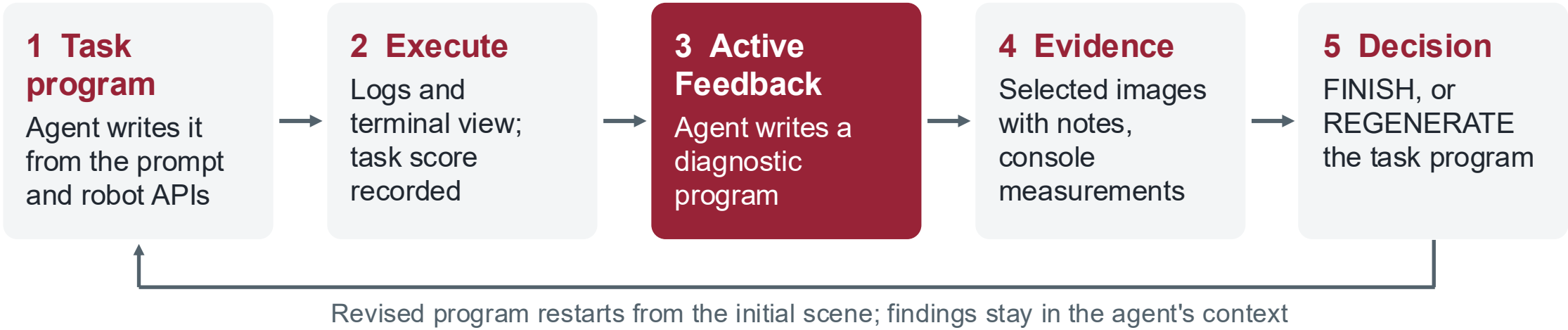
Contact is fragile, so one failure can have several causes.

After a failed door opening: bad grasp, bad commanded motion, or not enough platform capability?

Re-reading the same feedback cannot recover an observation that was never collected.
The agent has to go and get new evidence.

Lee et al., Fly-by-Code (2026 preprint, arXiv 2610.12549), Sections 1 and 3.1. Examples from the paper.

Active Feedback adds a diagnostic stage



View
Fly to a viewpoint that shows an unobserved outcome.

Probe
Act on the world to test a hypothesis about a failure cause.

Same robot APIs as the task program. The task score is recorded before diagnosis, so a diagnostic earns no credit. One prompt for all tasks; the program is written per attempt.

Lee et al., Fly-by-Code, Section 3 and Figure 2. Box labels paraphrase the paper.

Two recorded diagnostic rounds

View: did the tools land?

2-Objects Pick-and-place

Unresolved	Task program reported success. Gripper is empty. Is anything in the basket?
Diagnostic	Fly above the basket and shelf, look at the floor, return depth positions.
Evidence	Both tools sit near floor level.
Decision	Reject the reported success: REGENERATE.

Probe: why does the carton rotate?

Align

Unresolved	Pushes keep rotating the carton. Which contact point is best?
Diagnostic	Six push experiments at different contact points; measure rotation after each.
Evidence	Rotation per cm varies with contact location.
Decision	REGENERATE with a push-response model.

Planning-level probe (Payload): a lift test of a 1.2 kg basket load was refused; the next program regrouped under 1.1 kg and both loads were lifted.

https://fly-by-code.github.io/

Lee et al., Fly-by-Code, Figure 3 and Figure 6 (abridged recorded agent outputs).

Two recorded diagnostic rounds

View: did the tools land?



Probe: why does the carton rotate?

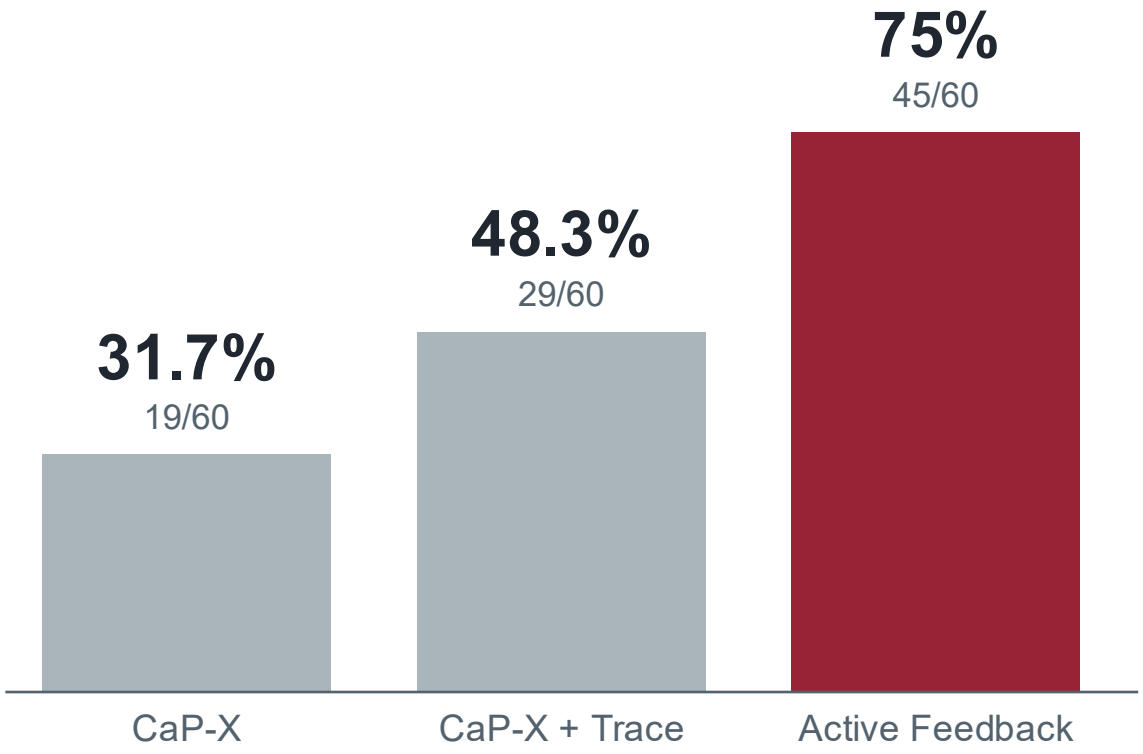


https://fly-by-code.github.io/

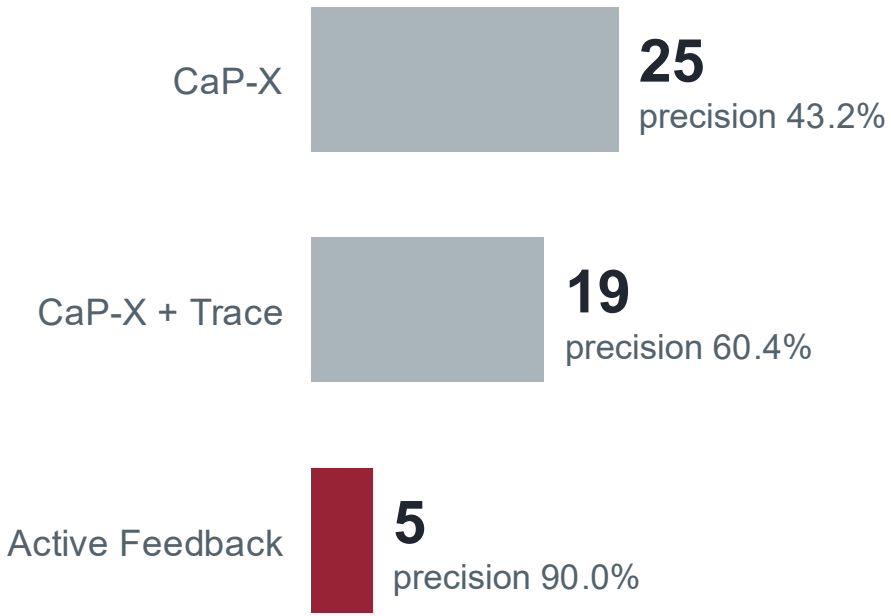
Lee et al., Fly-by-Code, Figure 3 and Figure 6 (abridged recorded agent outputs).

More evidence, fewer false completions

Final task success, 60 trials



False completions (lower is better)



Cost to keep in mind: the diagnostic stage adds inference and robot-interaction overhead (a limitation the paper states).

Lee et al., Fly-by-Code, Section 4.2, Figure 5, Table 1. 6 simulated tasks, 10 trials each; Claude Opus 5; one run + up to 10 revisions.

From simulation to a real aerial manipulator

4 / 5

2-Objects Pick-and-place, full task

Grasp 5/5, place 4/5, grasp 5/5, place 5/5

3 / 5

Cabinet, full task

Doors 5/5 and 4/5, grasp 4/5, place 3/5

Same task programs as in simulation, no source-code changes; 8 to 9 minutes of autonomous operation per run.

What the paper does not claim

Diagnosis adds inference and robot-interaction overhead.

A diagnostic check cannot fix flawed onboard perception or unreachable viewpoints.

Probing happens in simulation; no safety guarantee for physical probing.

No zero-shot claim for unseen environments or different robot bodies.

Lee et al., Fly-by-Code, Section 4.5, Table 2, and Section 6 (Limitations). 5 real-world trials per task.

Active Feedback through the lecture's harness lens

	SWE-agent trace	CaP-X	Fly-by-Code
Action contract	Edit commands with line ranges	Python over perception and control primitives	Same APIs, plus a diagnostic program
Observation contract	Bounded file view, lint error	stdout/stderr; image changes as text	Agent-chosen views and probes
Stopping and evaluation	submit; held-out tests	Success vs human-written code	FINISH; precision of declarations
Environment	Backup restored after a failed edit	Simulator; real Franka for CaP-RL	Probe in simulation; reset per attempt

Active Feedback changes the observation contract: the agent writes the program that produces its own evidence.

Instructional mapping. SWE-agent and mini-SWE-agent as in slides 11-33; CaP-X and Fly-by-Code as summarized in slides 34-43.

Is Active Feedback a harness component?

Yes, in what it changes

It changes what evidence reaches the model and what supports the decision to stop.

But it differs from the software examples

It acquires new evidence by acting. The diagnostic earns no task credit. Physical actions cannot be undone, so probes stay in simulation.

1. A reproduction script is cheap in software. What makes a diagnostic program cheap or costly on a robot?
2. Active Feedback uses extra inference and interaction. What comparison would separate better evidence from more budget?
3. Which parts of a diagnostic round would you keep as training data, and what would you check first?

Discussion prompts. Lecture synthesis from slides 27-33 and the Fly-by-Code limitations section.

Readings: embodied coding agents

CaP-X	Fu et al., ICML 2026, arXiv 2603.22435. Sections 3-4: the tiers, Takeaways 1-3, CaP-Agent0. Project page: capgym.github.io
Fly-by-Code	Lee et al., 2026 preprint, arXiv 2610.12549. Section 3, Figures 3 and 5, Table 1, Limitations. Project page: fly-by-code.github.io
ASPIRE	Lu et al., 2026, arXiv 2607.00272. Per-primitive multimodal traces, skill library, evolutionary search.

Fu et al. (2026); Lee et al. (2026); Lu et al. (2026). Project pages as linked in each paper.